3강. 식, 연산자, 명령문

- 식
- 연산자
- 타입(현) 변경
- 명령 실행 문

3-1. 식 - 계산문

계산 문이란, 변수나 값을 이용하여 여러가지 계산을 컴퓨터에 수행하게 하기 위한 문장이다.

계산 처리라고 한다면, 사칙연산 뿐만이 아니다.

변수에 값을 대입하는 것도 컴퓨터에게는 계산의 일종인 셈이다.

```
int a;
int b;
a = 20;
b = a + 5;  식 (expression)
System.out.println(a);
System.out.println(b);
```

3-1. 식의 구성요소

```
b = a + 5;
```

a, b, 5 : **피연산자** (operand)

+, = : **연산자** (operator)

아무리 복잡한 식이라도, **모든 식은 이 두가지 요소로 구성되어 있다.**

3-1. 식의 구성요소

1+5

연산자와 피연산자를 찾아보세요

3-1. 식의 구성요소

age = 30

연산자와 피연산자를 찾아보세요

3-1. 피연산자

피연산자란 “변수나 값" 이라고 생각하면 될까요?

-> 대체로는 맞지만, 좀 더 명확히 하기 위해 특별히 중요한 피연산자인 “**리터럴** (literal) " 에 대해 설명합니다.

3-1. 리터럴(literal)

- 피연산자 중에서도 숫자 5 나 문자열 "Hello World" 등, 소스코드에 기술되어 있는 값을 **리터럴** (literal) 이라고 합니다. 그리고 각각의 **리터럴은 데이터 형을 가지고 있습니다**.

리터럴의 종류	표기예	형
소수점이 없는 숫자 (정수)	30	int
소수점이 없는 숫자로 끝이 L 또는 l (큰 정수)	300000L	long
소수점 있는 숫자 (정밀도가 높은 소수)	30.5	double
소수점 있는 숫자로 끝이 F 또는 f (비교적 정밀도가 낮은 소수)	30.5F	float
true (참) 또는 false (거짓)	true	boolean
따옴표로 감싸여진 문자	'남'	char
이중따옴표로 감싸여진 문자열	"Java"	String

3-1. 리터럴(literal)

1, '1', "1"

같은 것인지 다른 것인지 생각 해 보세요

3-1. 리터럴(literal) – 정수 리터럴에 대한 응용

```
// 0x를 붙이면 16진수
int a = 0x11;    // 17

// 0b를 붙이면 2진수
int b = 0b0011; // 3

// 알기 쉽게 _ 를 붙이는 것도 가능
long price = 2_000_000; // 2,000,000 원
```

3-1. 이스케이프 시퀀스(escape sequence)

- String 형이나 char 형의 리터럴을 기술할 때, 가끔씩 사용되는 것이 **이스케이프 시퀀스** (escape sequence) 라고 불리우는 특수한 문자이다.
- "₩ (역슬래쉬) 와 한 문자" 의 총 두 문자로 구성된 기술방법으로, 이 두 문자로 특수한 1문자를 표현한다.

표기	의미
₩"	이중따옴표 (")
₩'	따옴표 (')
₩₩	역슬래쉬 또는 원 기호 (₩)
₩n	개행 (제어문자)

3-1. 이스케이프 시퀀스(escape sequence) – 이스케이프 시퀀스 미사용 예(에러)

```
System.out.println("내가 좋아하는 기호는 쌍따옴표(")입니다");
```

Java 에서는 **두개의 이중따옴표로 둘러 쌓인 부분** 을 문자열로 취급합니다.

<잠깐>: 에러를 수정 해 봅시다.

3-1. 계산(evaluation)

Java 가 식에 따라 계산 처리를 하는 것을, 식의 **계산** (evaluation) 이라고 한다.

Java는 3가지 단순한 원칙에 의해 계산을 수행 합니다.

I. 계산 결과로의 치환 법칙
II. 우선 순위의 법칙
III. 결합 규칙의 원칙

3-1. 계산(evaluation) – 계산 결과로의 치환 법칙

연산자는 주변의 피연산자의 정보를 사용해 계산을 하고, 계산 결과로 치환 한다.

1 + 5 - 3

3-1. 계산(evaluation) – 우선 순위의 원칙

식에 연산자가 여러개 있는 경우, **Java 에서는 정해진 우선순위가 높은 연산자 부터 순서대로** 계산 합니다.

1+5*3 (1+5)*2

3-1. 계산(evaluation) – 결합 규칙의 원칙

식 안에 같은 우선순위 그룹에 속한 연산자가 여러개 있을 경우, **연산자별로 정해진 "방향" 부터 순서대로** 계산 한다

10+5+2 a=b=8

3-2. 연산자

일정한 규칙을 가지고 계산하는 것으로서, 우리가 일상생활에서 사용하고 있는 덧셈, 뺄셈, 곱셈, 나눗셈을 비롯하여 프로그램에서는 자동 증감, 동등 비교, 비트 연산 등 많은 연산자가 있다.

다양한 연산자 : 산술, 문자열 결합, 대입, 증감, 동등 비교, 논리, 기타 등등

3-2. 연산자 – 산술 연산자

➢ 산술 연산자는 더하기(+), 빼기(-), 곱하기(*), 나누기(/), 나머지(%)로 총 5개이다.

연산자	기능	계산의 방향	계산의 예
+	더하기	좌->우	3 + 5 -> 8
-	빼기	좌->우	10 - 3 -> 7
*	곱하기	좌->우	3 * 2 -> 6
/	나누기 (정수 연산에서는 몫)	좌->우	3.2 / 2 -> 1.6 9 / 2 -> 4
%	나누기 한 나머지	좌->우	9 % 2 -> 1

산술 연산자 특징

- 피연산자가 정수 타입(byte, short, char, int)이면 연산의 결과는 int 이다.
- 피연산자가 정수 타입이고 그중 하나가 long 타입이면 연산의 결과는 long이다.
- 피연산자 중 하나가 실수 타입이면 연산의 결과는 실수 타입이다.

3-2. 연산자 – 문자열 결합 연산자

연산자	기능	계산의 방향	계산의 예
+	문자열의 결합	좌->우	“안녕" + “하세요" -> “안녕하세요"

3-2. 연산자 – 대입 연산자

➢ 대입 연산자는 우측 피연산자의 값을 좌측 피연산자인 변수에 대입한다.

연산자	기능	계산의 방향	계산의 예
=	우변을 좌변에 대입	우->좌	a = 10 -> a (10)
+=	좌변과 우변을 더해 좌변에 대입	우->좌	a += 2 -> a (a = a + 2 와 동일)
-=	좌변에서 우변을 빼서 좌변에 대입	우->좌	a -= 2 -> a (a = a - 2 와 동일)
*=	좌변과 우변을 곱해 좌변에 대입	우->좌	a *= 2 -> a (a = a * 2 와 동일)
/=	좌변에서 우변을 나누어 좌변에 대입	우->좌	a /= 2 -> a (a = a / 2 와 동일)
%=	좌변에서 우변을 나누어 그 나머지를 좌변에 대입	우->좌	a %= 2 -> a (a = a % 2 와 동일)
+=	좌변의 뒤에 우변을 결합 하여 대입	우->좌	a += "팀" -> a (a = a + "팀" 과 동일)

3-2. 연산자 – 증감 연산자

➢ 증감 연산자(++, --)는 변수의 값을 1 증가시키거나 1감소시키는 연산자이다.

연산자	기능	계산의 방향	계산의 예
++	값을 1 증가 시킴	좌->우	a++ -> a (a = a + 1 또는 a += 1 과 동일)
--	값을 1 감소 시킴	좌->우	a-- -> a (a = a - 1 또는 a -= 1 과 동일)

```
int a = 10;
a++;
System.out.println("a = " + a);

a--;
System.out.println("a = " + a);
```

3-2. 연산자 – 증감 연산자

- 여러 개의 연산자가 포함되어 있는 연산식에서는 증감 연산자의 위치에 따라 결과가 달라진다. 증감 연산자가 변수 앞에 있으면 변수를 1 증가 또는 1 감소시킨 후에 다른 연산을 수행하고, 증감 연산자가 변수 뒤에 있으면 모든 연산이 끝난 후에 1 증가 또는 1 감소시킨다.

```
int a = 10;
int b = 10;

System.out.println(++a);                    // a: 11
System.out.println(b++);                    // b: 10
System.out.println(b);                      // b: 11

int x = 10;
int y = 10;
int result1 = ++x + 10;
int result2 = y++ + 10;

System.out.println("result1 = " + result1);  // x: 11
System.out.println("result2 = " + result2);  // y: 10
System.out.println("y = " + y);              // y: 11
```

3-3. 타입(형) 변환(Cast)

```
double d = 3;
String s = "베스트 " + 3;

System.out.println("s = " + s);      // s: 베스트 3
```

타입(형)이 맞지 않는 대도 에러가 나지 않는 이유는, **java가 식을 계산하는 과정에서 자동적으로 타입(형)을 변환하고 있기** 때문입니다.

1. 대입시에 자동 타입 변환
2. 명시적인 강제 타입 변환
3. 연산식에서 자동 타입 변환

3-3. 타입(형) 변환 – 자동 타입 변환

- 자동 타입 변환(promotion)은 말 그대로 자동으로 타입 변환이 일어나는 것을 말한다. 자동 타입 변환은 값의 허용 범위가 작은 타입이 허용 범위가 큰 타입으로 대입될 때 발생한다.

자동타입변환

큰 허용 범위 타입 = 작은 허용 범위 타입

- 기본 타입 허용 범위 순으로 나열

byte < short, char < int < long < float < double

- int 타입이 byte 타입보다 허용 범위가 더 크기 때문에 자동 타입 변환

```
byte byteValue = 10;
int intValue = byteValue;          // 자동 타입 변환됨

float f = 3;
double d = f;
System.out.println(d);             // 자동 타입 변환됨

int i = 3.2;                       // 에러(허용범위가 작음)
```

3-3. 타입(형) 변환 – 강제 타입 변환

- 큰 허용 범위를 작은 허용 범위 타입으로 쪼개어서 저장하는 것을 강제 타입 변환(casting)이라고 한다. 강제 타입 변환은 캐스팅 연산자로 괄호()를 사용하며, 괄호 안에 들어가는 타입은 쪼개는 단위이다.

강제타입변환

작은 허용 범위 타입 = (작은 허용 범위 타입) 큰 허용 범위 타입

- 기본 타입 허용 범위 순으로 나열

byte < short, char < int < long < float < double

- 사용 예

```
int iAge;
iAge = 32;                    // 타입에 맞게 사용
iAge = 3.2;                   // 에러(허용범위가 작음)
iAge = (int) 3.2;             // 강제 타입 변환( 3: 정수 부분 저장 )
```

3-3. 타입(형) 변환 – 연산식에서 자동 타입 변환

- 산술 연산자에 의한 계산이 수행될 때 "좌우 피연산자는 동일한 타입" 이 원칙이다.

```
int  iValue = 10;
double dValue = 5.5;
double result = iValue + dValue;    // 10.0 + 5.5
```

- 위처럼 int 타입과 double 타입이 연산하는 경우에도 int 타입 피연산자가 double 타입으로 자동 변환되고 수행한다.

- 정수 연산 예

```
int x = 1;
int y = 2;
double result = x / y;
System.out.println("result = " + result);        // 0.5가 출력될까요?
```

- 자바에서 정수 연산의 결과는 항상 정수가 되기 때문에 x / y 는 0 이 나오고 double 타입 변수에 저장하므로 0.0의 결과가 나온다.

3-3. 타입(형) 변환 – 연산식에서 자동 타입 변환

수치형 끼리의 연산시 타입 변환 규칙

- 한쪽 피연산자가 double 이면, 다른 쪽도 double로 타입 변환 됨
- 한쪽 피연산자가 float 이면, 다른 쪽도 float로 타입 변환 됨
- 한쪽 피연산자가 long 이면, 다른 쪽도 long로 타입 변환 됨
- 한쪽 피연산자가 int 이면, 다른 쪽도 int로 타입 변환 됨
- short 이나 byte 피연산자는 int로 타입 변환 됨

```
double d = 8.5 / 2;
long l = 5 + 2L;
System.out.println("d = " + d);        // 4.25
System.out.println("l = " + l);        // 7
```

문자열의 연결

- 한쪽 피연산자가 String 이면, 다른 쪽도 String으로 타입 변환되어 연결됨

```
String msg = "내 키는 " + 184;
System.out.println(msg);               // 내 키는 184
```

3-4. 명령 실행 문

- 자바가 준비해 준 여러가지 명령을 사용하는 것
- System.out.println 등

명령(화면출력) 실행문

```
String name = "홍길동";
String message;
message = name + "씨, 안녕하세요!!";
System.out.println(mesage);
```

```
String name = "홍길동";
System.out.println(name + "씨, 안녕하세요!!");
```

인자(argument, 매개변수)

```
String name = "홍길동";
System.out.println(name + "씨, 안녕하세요!!");
```

3-4. 명령 실행 문

개행 없이 화면에 출력하는 명령

```
String name = "홍길동";
System.out.print("내 이름은 ");
System.out.print(name);
System.out.print("입니다.);
```

더 큰 숫자를 대입하는 명령

```
int a = 5;
int b = 3;
int m = Math.max(a, b);
System.out.println("비교: " + a + "와 " + b + " 중 큰 쪽은 " + m);
```

String타입을 int 타입으로 변환하는 명령

```
String age = "25";
int n = Integer.parseInt(age);
System.out.println("당신은 내년에 " + (n + 1) + " 살이 됩니다.");
```

3-4. 명령 실행 문

랜덤한 수를 생성하는 명령

```
int r = new java.util.Random().nextInt(90);
System.out.println("랜덤한 수 " + r);
```

키보드로부터 입력을 받는 명령

```
System.out.println("당신의 이름을 입력해주세요!");
String name = new java.util.Scanner(System.in).nextLine();

System.out.println("당신의 나이를 입력해주세요!");
int age = new java.util.Scanner(System.in).nextInt();

System.out.println("반갑습니다. " + age + "살의 " + name + "님");
```

3-5. 요점 정리

수식

- 수식은 연산자와 피연산자로 구성되어 있다.
- 리터럴에도 타입(형)이 있어 기술 방법에 의해 결정된다.
- 연산자가 계산되면, 그 연산자와 피연산자는 결과로 변한다.
- 연산자는 우선 순위와 결합 규칙에 따라 계산된다.

타입 변환

- 큰 타입에 작은 데이터를 대입하면 자동적으로 타입 변환되어 대입된다.
- 작은 타입에 큰 데이터를 대입하면 강제 타입 변환(casting)하여 대입 할 수 있다.
- 수식의 계산 시 큰 데이터에 맞게 자동적으로 타입이 변환된다.

명령 실행

- 자바에서 준비되어 있는 여러가지 명령을 실행할 수 있다.

3-6. 예제 1

```java
public class Main {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;
        String answer = "x+y는" + x + y;
        System.out.println(answer);
    }
}
```

“x+y는 15” 가 표시되도록 수정 하시오

3-6. 예제 2

다음 중 문법이 올바른 것을 모두 고르시오.

1. int x = 3 + 5.0;
2. double d = 2.0F;
3. int i = “5”;
4. String s = 2 + “ 명째";
5. byte b = 1;
6. double d = true;
7. short s = (byte)2;

3-6. 예제 3

다음 내용의 프로그램을 작성 하시오. **(java_03_2_ex2_calEx)**

1. 사용자로부터 숫자를 입력(Scanner) 받으세요.
2. 입력 받은 숫자가 3의 배수인지 판단 후 결과를 화면에 표시하세요.

3-6. 예제 4

다음 내용의 프로그램을 작성 하시오.

1. 화면에 “점을 보세요" 라고 표시합니다.
2. 화면에 “이름을 입력해 주세요" 라고 표시합니다.
3. 키보드로부터 문자열을 입력 받아, String 형 변수 name 에 넣습니다.
4. 화면에 “나이를 입력 해 주세요" 라고 표시합니다
5. 키보드로부터 문자열 입력을 받아, String 형 변수 ageString 에 넣습니다
6. 변수 ageString 의 내용을 int 형으로 변환하고, int 형 변수 age 에 대입합니다
7. 0 부터 3 까지의 난수를 생성해, int 형 변수 fortune 에 대입합니다
8. fortune 의 수치를 증가연산자로 1 증가시켜, 1 부터 4 까지의 난수로 합니다.
9. 화면에 “점꾀가 나왔습니다!” 라고 표시합니다.
10. 화면에 “(나이)살의 (이름)씨, 당신의 운세번호는 (난수) 입니다" 라고 표시합니다. (나이) 에는 변수 age를, (이름) 에는 변수 name 을, 그리고 (난수) 에는 8. 에서 만든 난수의 숫자를 표시합니다.
11. 화면에 “1:대박 2:중박 3:보통 4:망" 이라고 표시합니다